Building on Codex

EECS 498 AASE, Lab 5

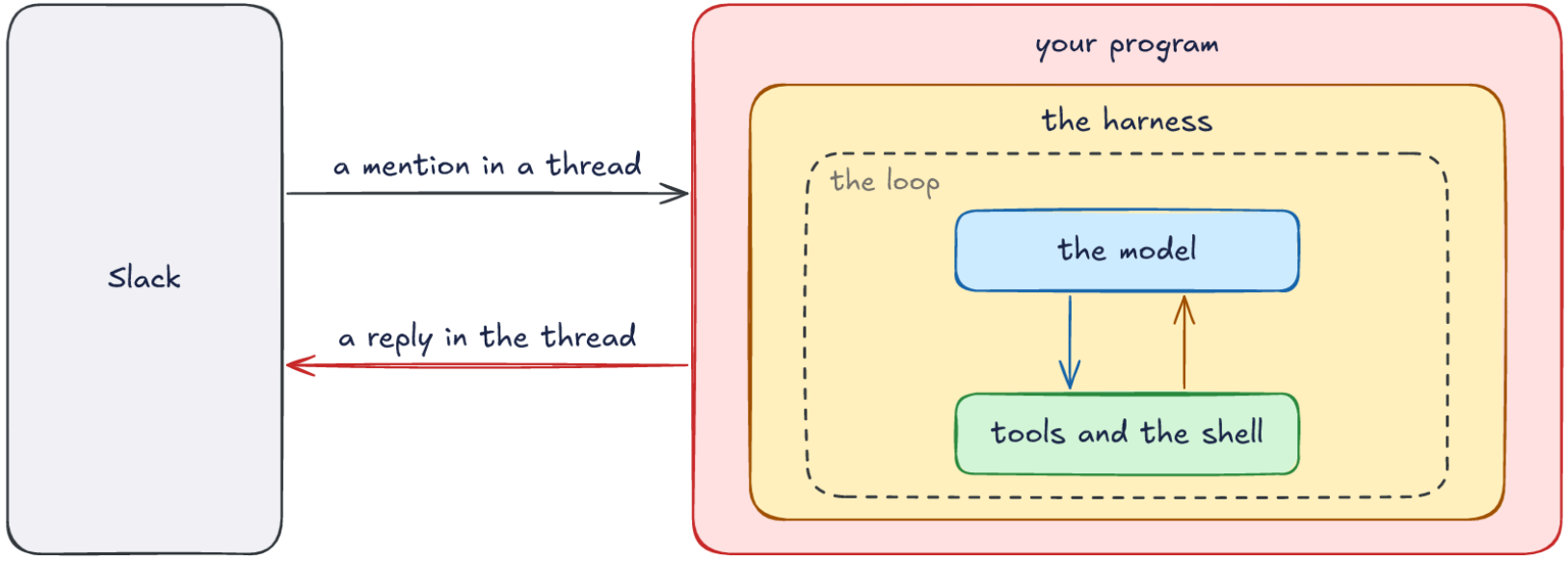
Today's schedule

- The problem
- The app-server
- Custom tools
- A toy example

A chat bot in Slack

- Your team mentions the bot in Slack
- The bot replies in the thread

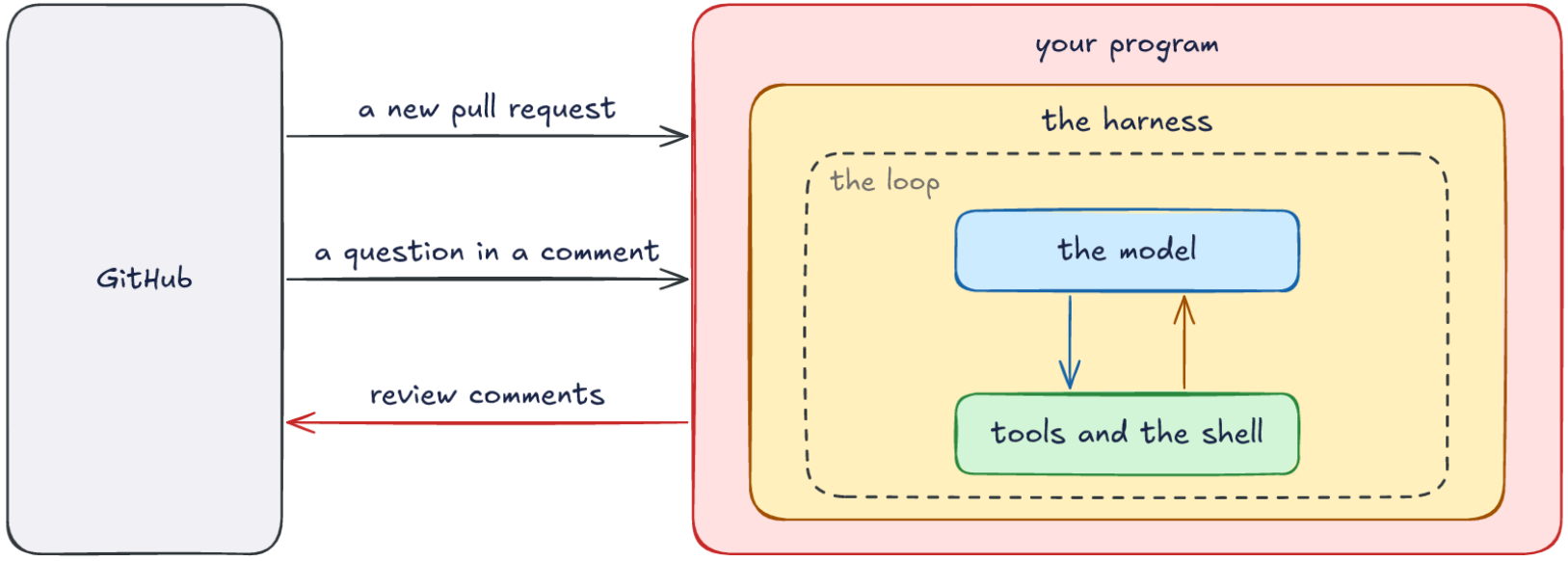
The chat bot and its harness



A review bot on every pull request

- Every new pull request gets a review
- Each new push gets reviewed again
- People ask the bot questions in comments

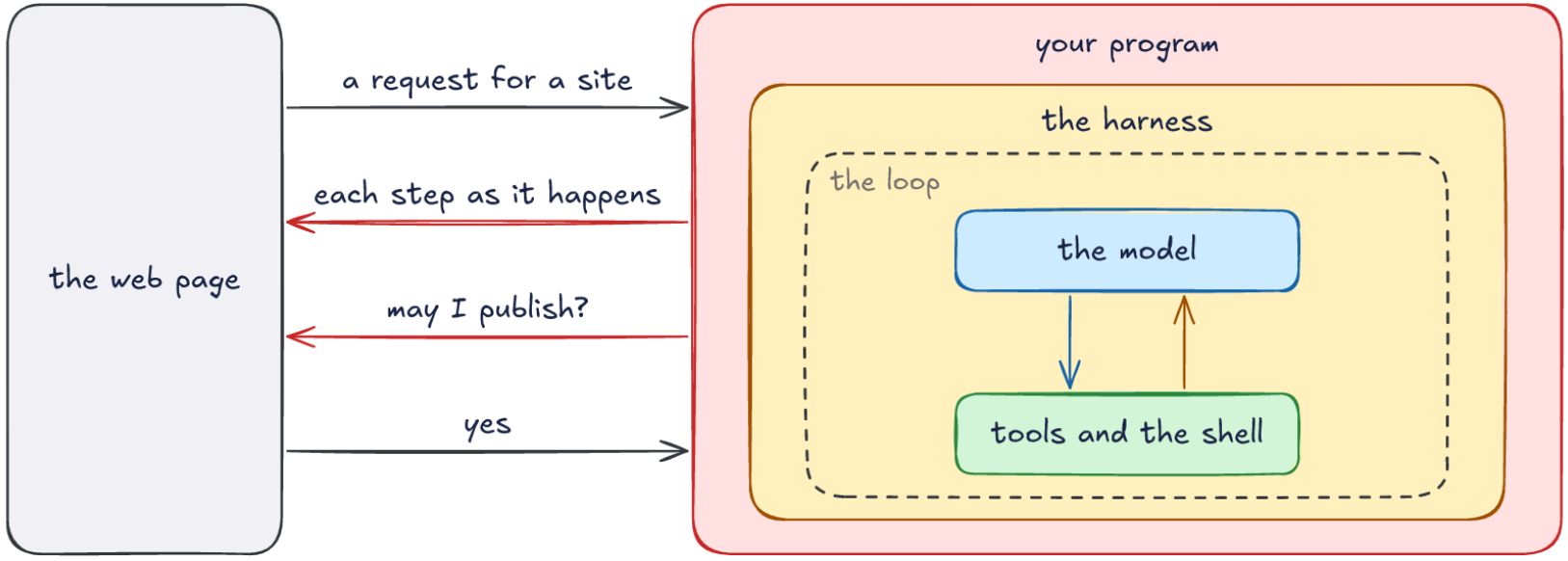
The review bot and its harness



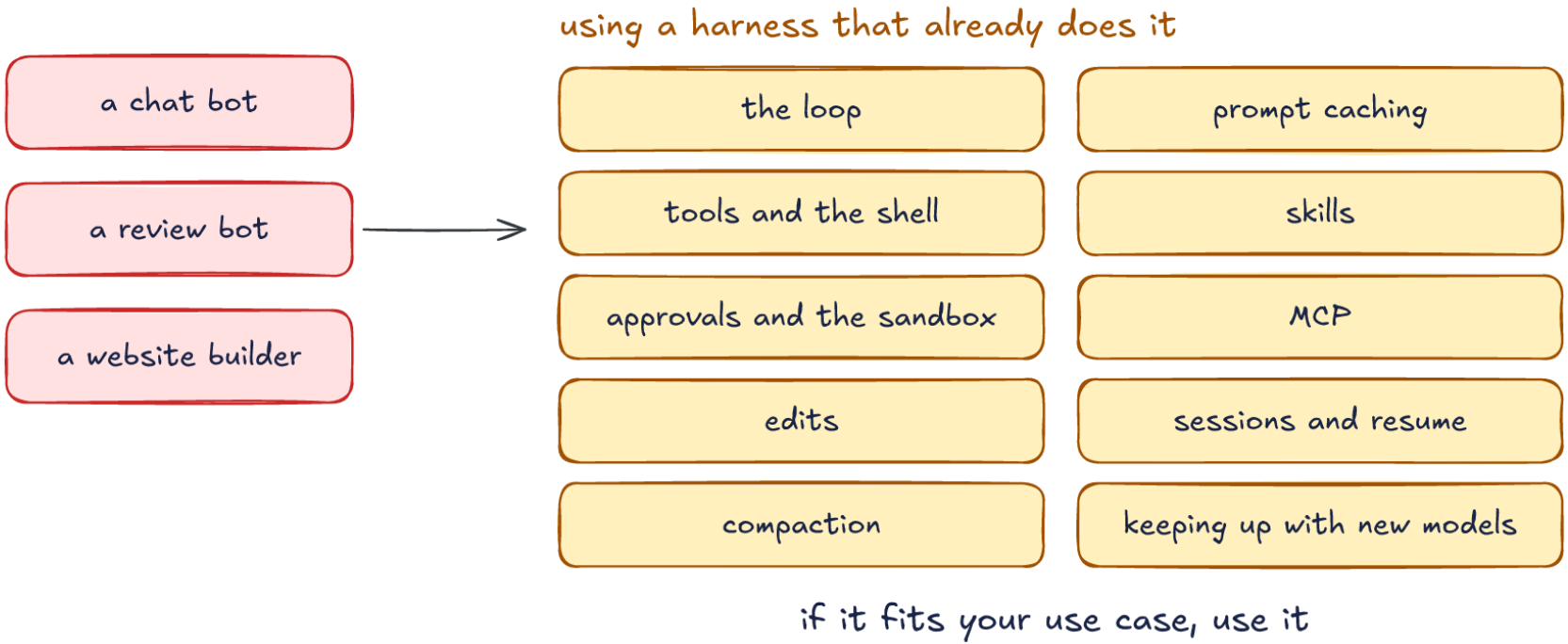
A website builder in the browser

- You describe a site in a chat
- You watch the site take shape
- The builder publishes when you approve

The website builder and its harness



Building a harness or using one

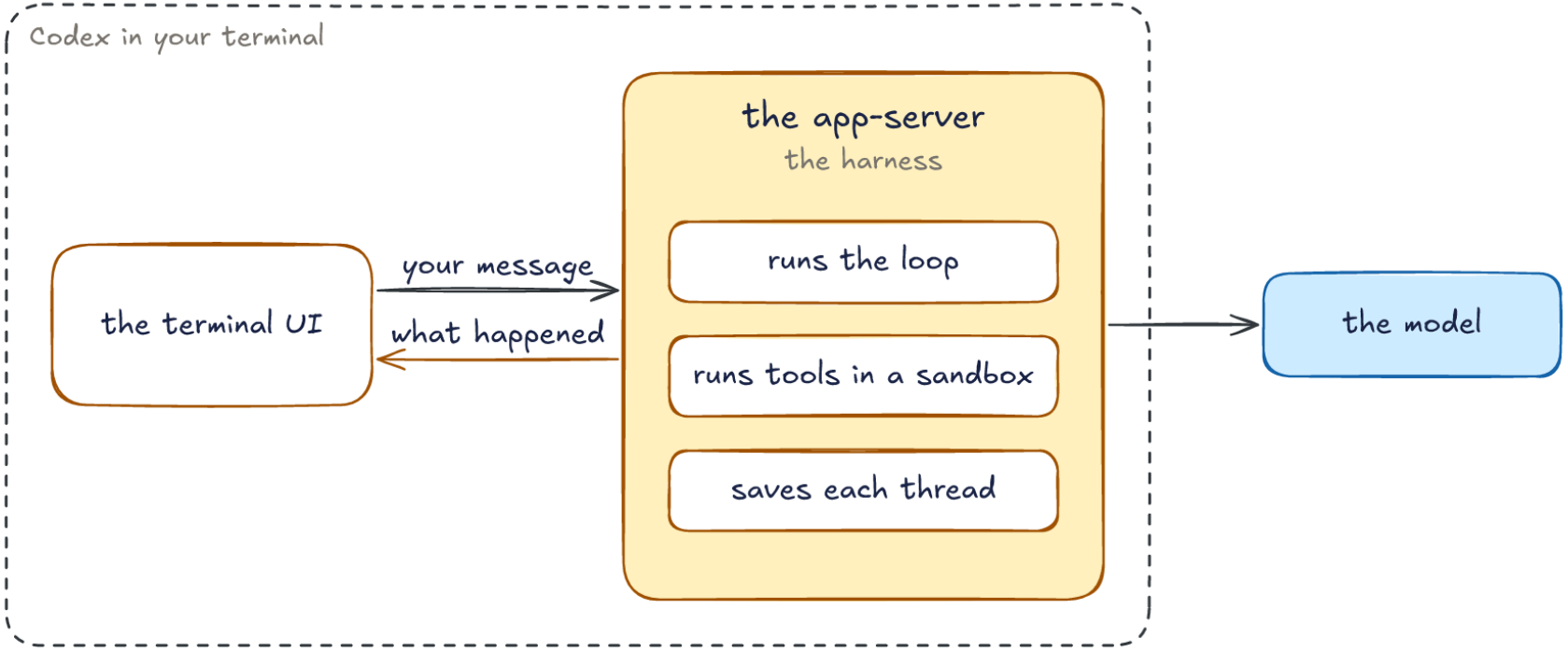


if it fits your use case, use it

Part 1

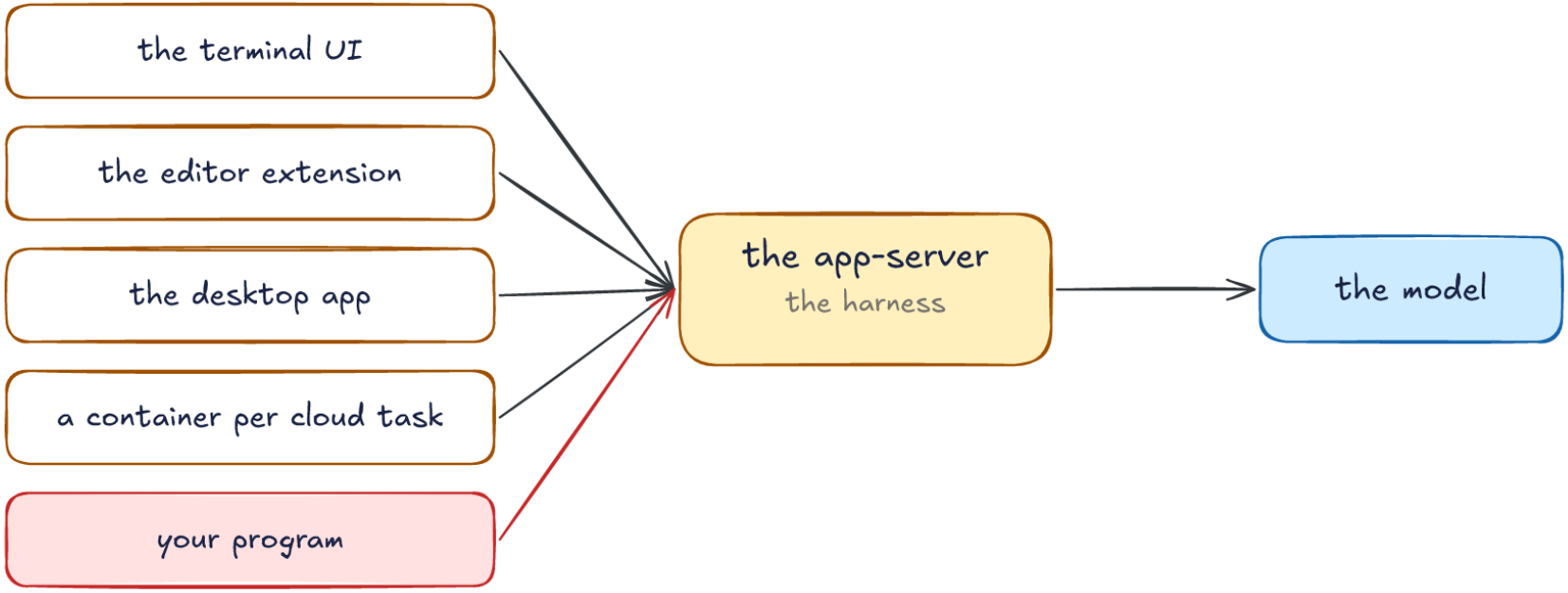
The app-server

What the app-server is



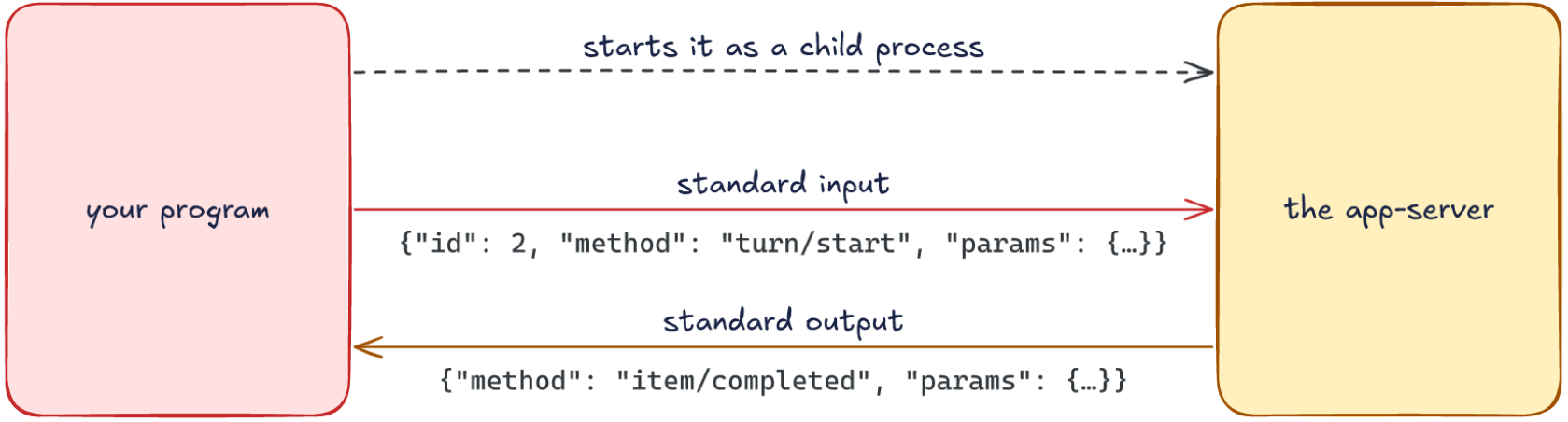
the terminal UI is one client of the app-server

Clients of the app-server



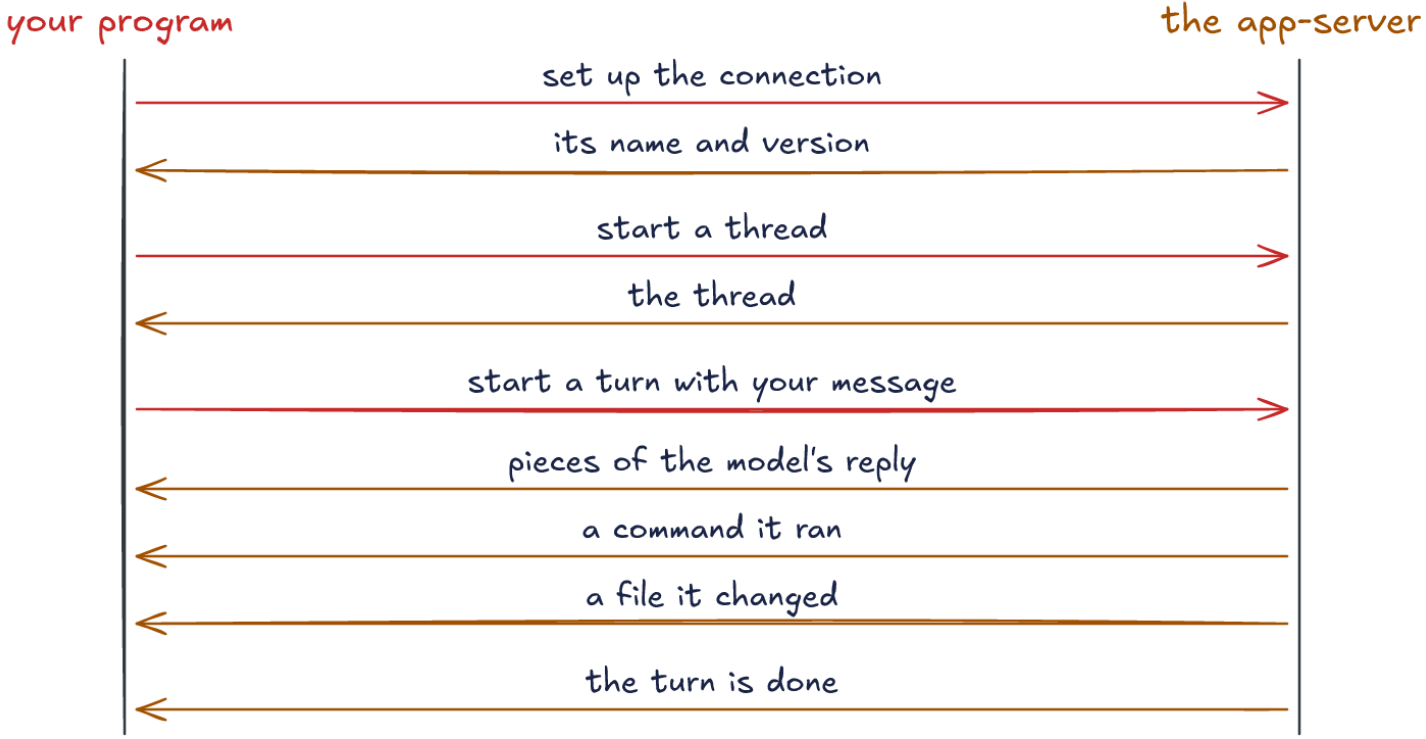
every Codex app is a client, and yours can be one too

How your program talks to the app-server

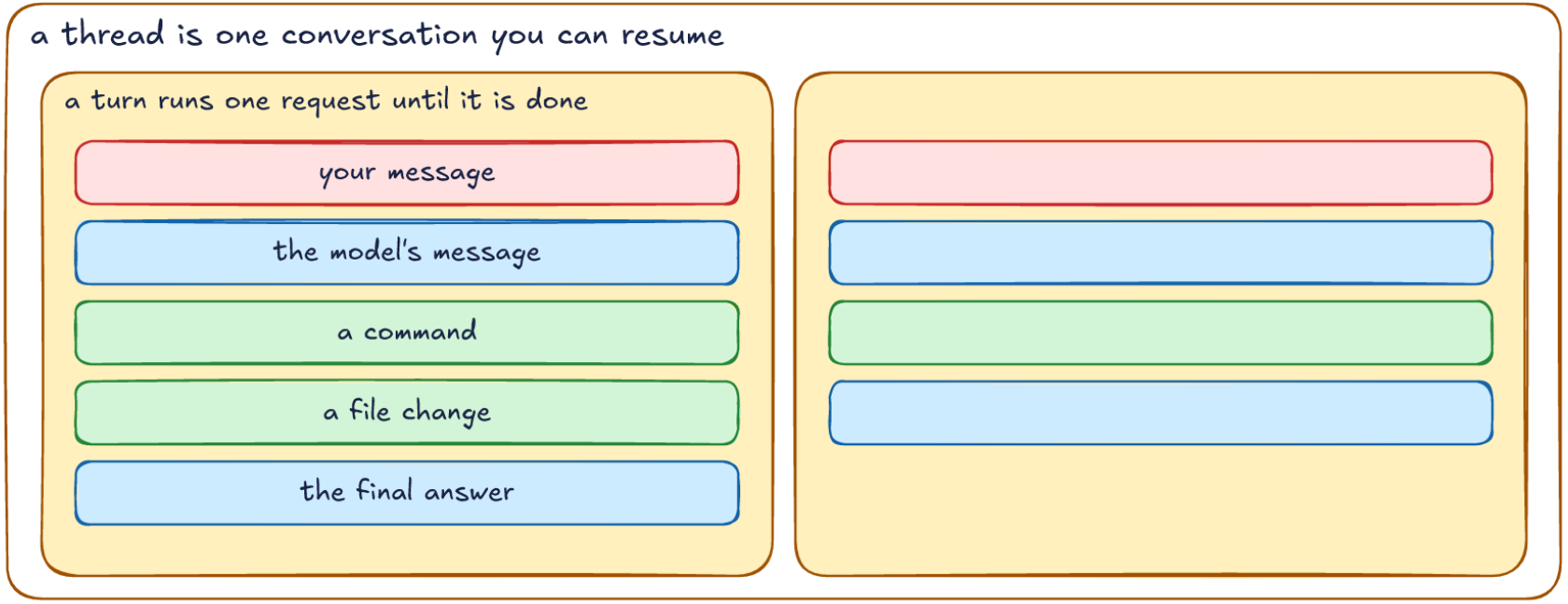


one line of JSON per message

One conversation over the app-server

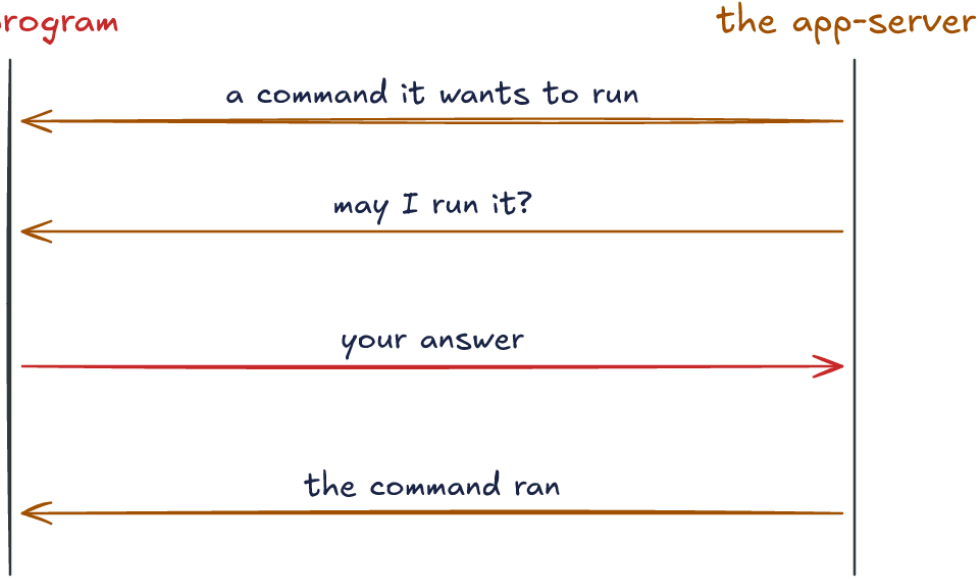


Threads, turns and items



each item starts, streams in pieces, then completes

An approval in the other direction



accept

accept for the session

decline and keep the turn going

cancel and stop the turn

a file change asks the same way, but its diff came in the earlier item

Steering and stopping a turn



What each side keeps

Codex keeps

- the loop
- tools and the sandbox
- edits
- compaction and caching
- skills and MCP
- sessions

your program keeps

- where people talk to it
- your own tools
- who approves
- memory
- schedules

the protocol is the line between them

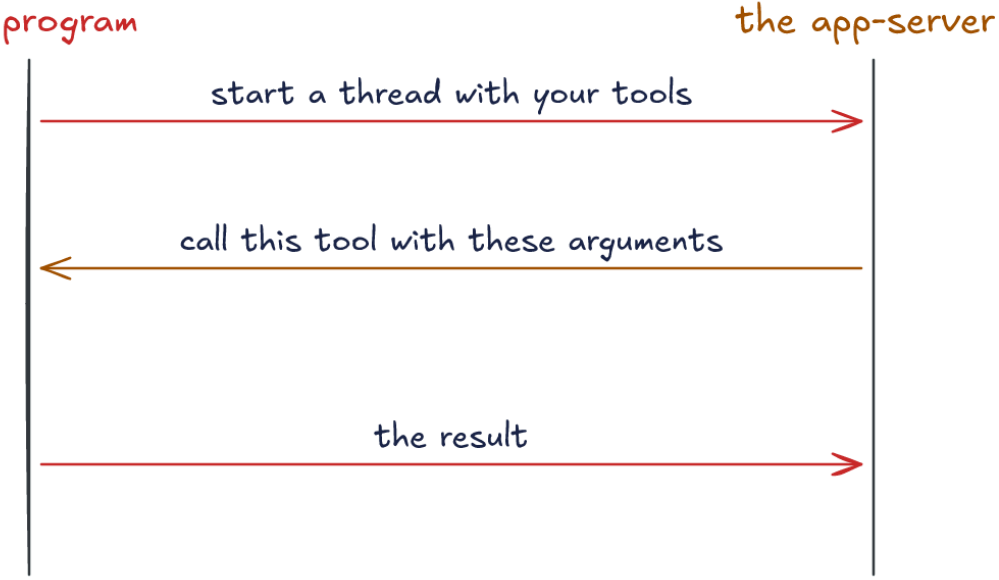
Part 2

Custom tools

A custom tool call

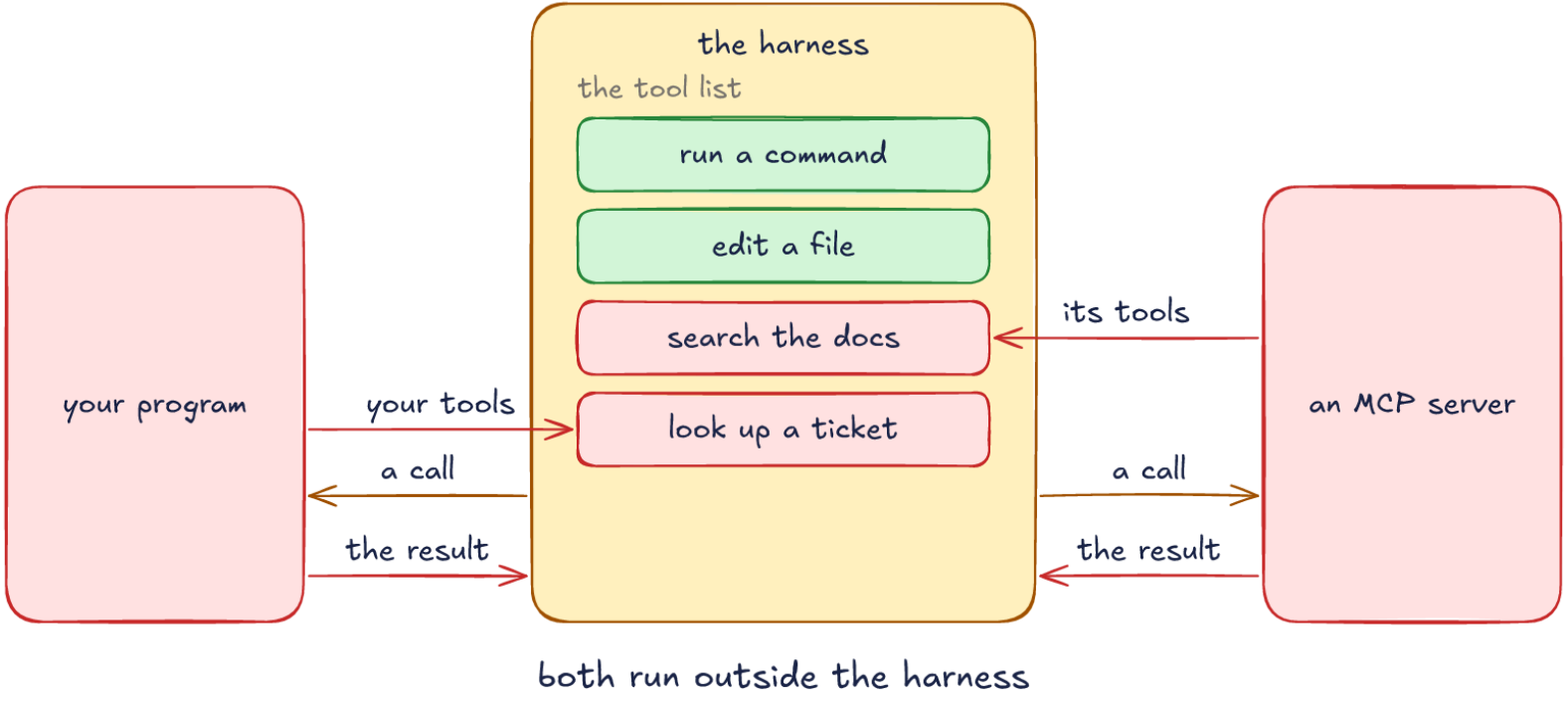
each tool has a name
a description and an input schema

your program runs it

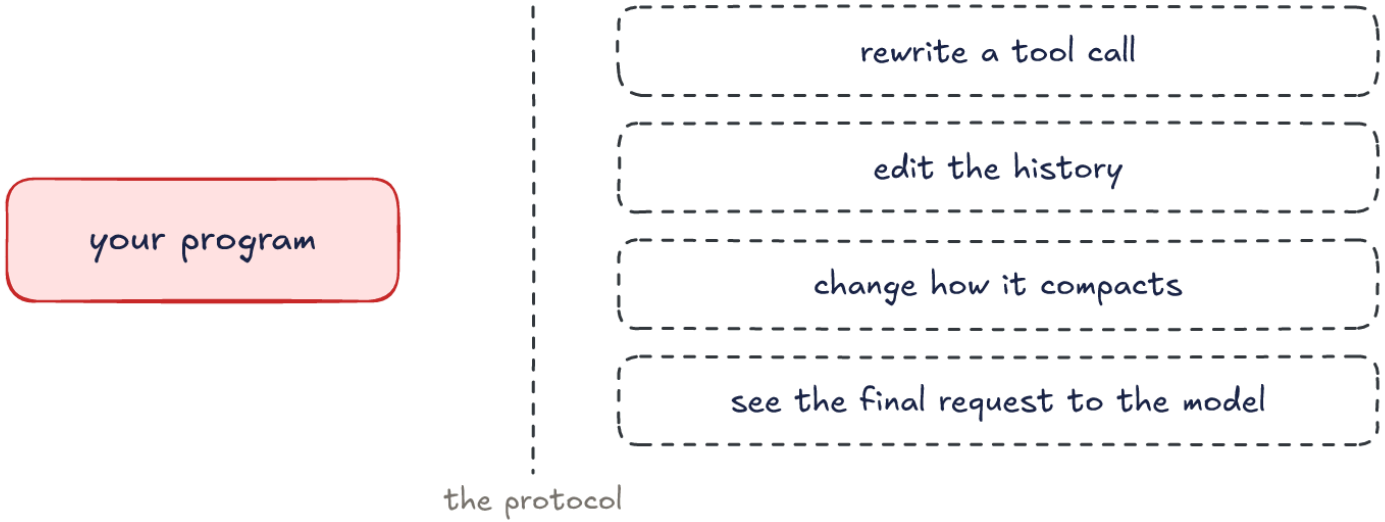


the model reads the result and keeps going

Custom tools and MCP



What you cannot change

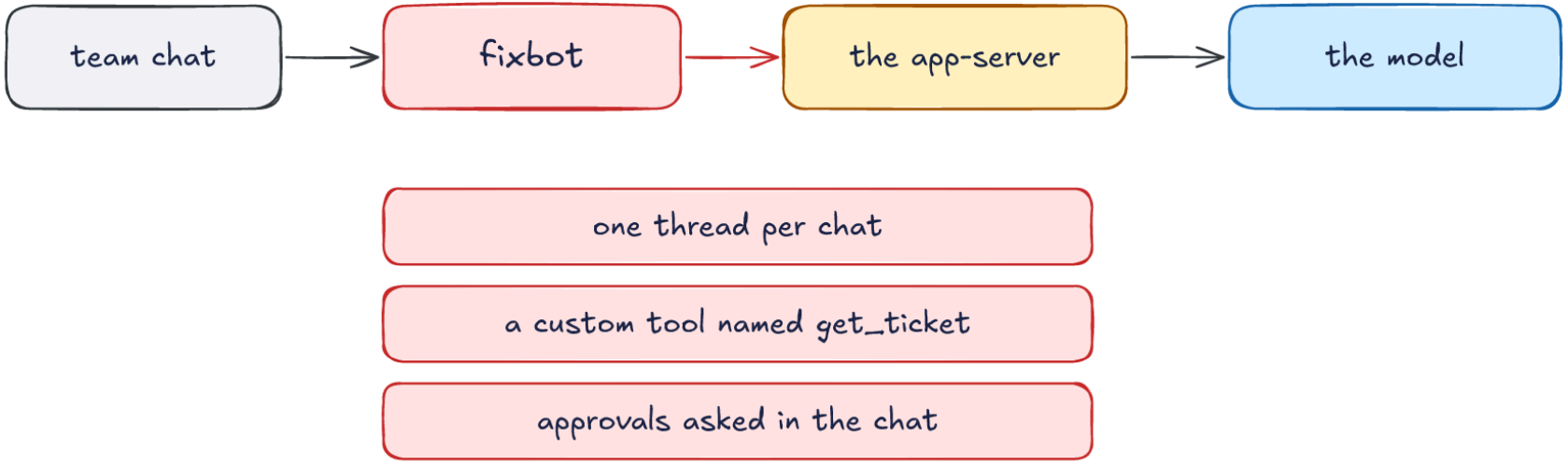


build your own loop when you need that control

Part 3

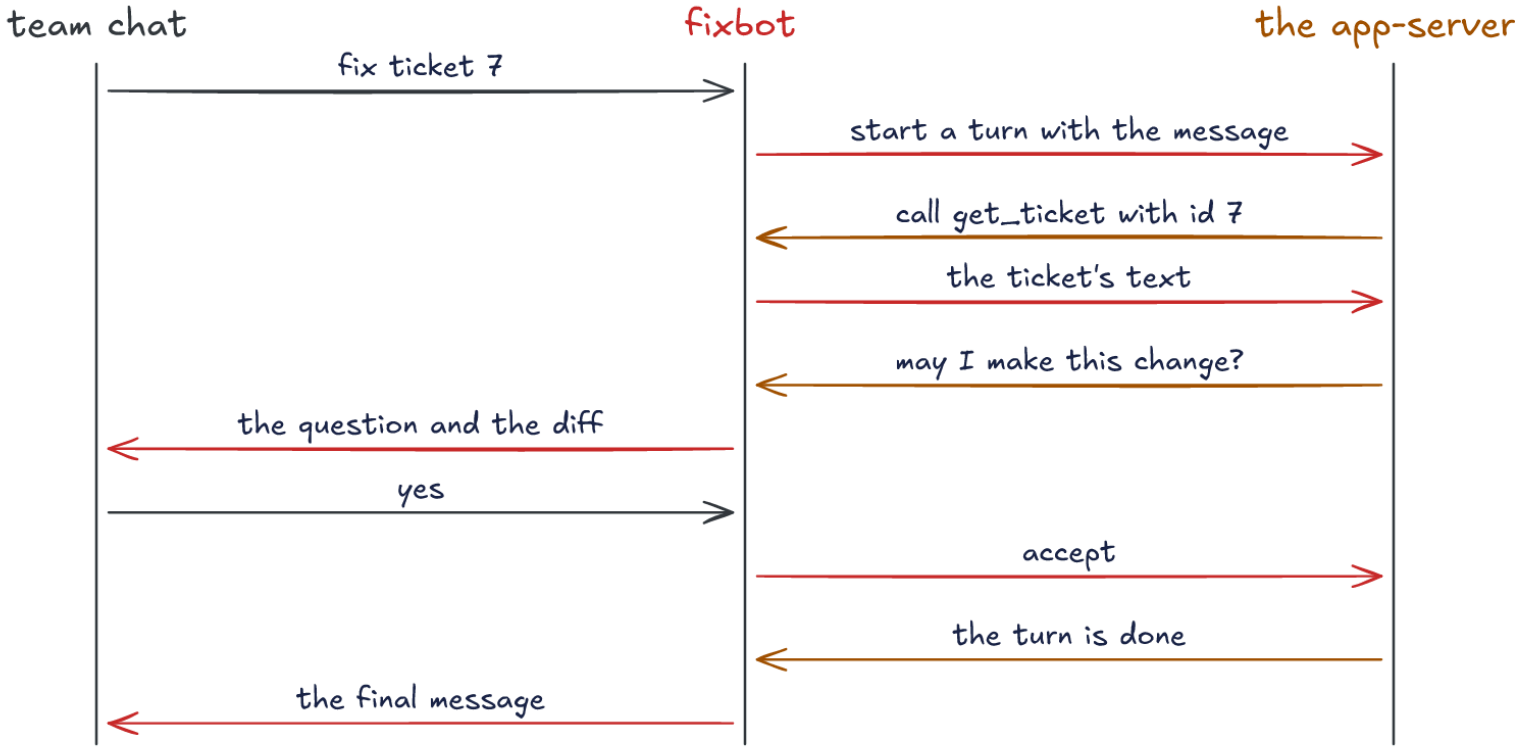
A toy example

A toy chat bot on Codex



Codex does the rest, and fixbot is one short Python file

One request through fixbot



The code behind fixbot

the loop that answers Codex

```
while True:
    msg = receive()
    method, params = msg.get("method"), msg.get("params", {})
    if method == "item/tool/call":
        text = TICKETS.get(params["arguments"]["id"], "no such ticket")
        send({"id": msg["id"], "result": {"contentItems": [{"type": "inputText", "text": text}],
                                          "success": True}})
    elif method == "item/commandExecution/requestApproval":
        send({"id": msg["id"], "result": {"decision": "accept"}})
    elif method == "item/fileChange/requestApproval":
        ok = ask_chat("May I make this change?\n" + diffs[params["itemId"]])
        send({"id": msg["id"], "result": {"decision": "accept" if ok else "decline"}})
    elif method == "item/completed" and params["item"]["type"] == "agentMessage":
        last = params["item"]["text"]
    elif method == "turn/completed":
        chat("fixbot", last)
        break
```

What to take away

- Most systems can use an existing harness
- Codex's harness runs as the app-server
- Your program never needs to know the loop
- Custom tools plug in like MCP tools

Sources

OpenAI on unlocking the Codex harness
openai.com/index/unlocking-the-codex-harness

OpenAI on Codex as a platform
openai.com/index/codex-as-a-platform

The Codex app-server docs
learn.chatgpt.com/docs/app-server

The Codex source
github.com/openai/codex

Questions

AASE Lab 5, Building on Codex